МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования
Гомельский государственный технический университет имени П. О. Сухого

Факультет автоматизированных и информационных систем
Кафедра «Информационные технологии»

специальность 6-05-0611-01 «Информационные системы и технологии»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту

по дисциплине «Программирование сетевых приложений»

на тему «Стресс-тестирование портфеля ценных бумаг
с учётом прогноза»

Исполнитель: студентка ИТД-41
Бокая Д. В.
Руководитель: доцент
Комраков В. В.

Дата проверки: _____
Дата допуска к защите: _____

Оценка работы: _____

Подписи членов комиссии по защите
курсового проекта: _____

Гомель 2026

СОДЕРЖАНИЕ

Введение 4
1 Обзор методов и программно-технического обеспечения 5
1.1 Подходы к оценке риска портфеля 5
1.2 Стрессовые сценарии и обоснование метода 6
1.3 Геометрическое броуновское движение и зависимости 7
1.4 Технологии распределённого вычисления 8
2 Проектирование и реализация программы 10
2.1 Архитектура, структура и исходные данные 10
2.2 Реализация математических алгоритмов 14
2.3 Сетевое взаимодействие и хранение результатов 17
2.4 Запуск процессов и графический интерфейс 20
3 Верификация и валидация программы 25
3.1 Верификация математических алгоритмов 25
3.2 Верификация сетевого обмена и хранения 26
3.3 Валидация пользовательских и отказных сценариев 27
3.4 Апробация приложения и анализ измерений 29
Заключение 31
Список использованных источников 32
Приложение А Условия и результаты испытаний 33
Приложение Б Листинг программы и тестов 37
Приложение В Руководство пользователя 80
Приложение Г Руководство программиста 95
Приложение Д Руководство системного программиста 106

ВВЕДЕНИЕ

Финансовый портфель объединяет инструменты, цены которых меняются под действием разных факторов. Положительный прогноз не исключает падения акций, валютного шока или роста колебаний стоимости облигаций. Оценка устойчивости требует рассматривать распределение годовых убытков. Стресс-тестирование позволяет явно задать неблагоприятные условия и сравнить их с базовым сценарием при одинаковом составе активов.

Моделирование Монте-Карло формирует случайные траектории и рассчитывает конечную стоимость портфеля. Эмпирический квантиль *VaR* характеризует порог, превышаемый в заданной доле модельных исходов. Он удобен для сравнения, но не описывает максимальный убыток. Поэтому программа также показывает гистограмму, вероятность потери и среднее значение в хвосте распределения.

С увеличением числа активов, шагов и траекторий растёт вычислительная нагрузка. Независимые траектории можно распределить между процессами и потоками. Сетевая реализация должна учитывать обрывы, повторы, недоступность узлов, отмену и сохранность истории. Ответ части вычислителей не должен подменять результат полного эксперимента.

Цель проекта – разработать распределённое приложение для стресс-тестирования портфеля акций, облигаций и валютных позиций с учётом заданного прогноза и расчётом *VaR95* и *VaR99*. Объект исследования – обработка сценариев стоимости портфеля. Предмет – генерация коррелированных траекторий, распределение партий и контроль полноты сетевого расчёта.

Задачи работы: определить параметры и ограничения; реализовать геометрическое броуновское движение и разложение корреляционной матрицы; организовать *TCP*-обмен и многопоточность; агрегировать, сохранять и экспортировать потери; создать интерфейс сценариев; провести математические и сетевые испытания. Решения должны позволять повторить опыт с сохранёнными параметрами и начальным числом генератора.

Приложение *ASTER* создано на *C#* со стандартной библиотекой *.NET* 8. Браузер обращается к *HTTP*-шлюзу, который служит *TCP*-клиентом управляющего процесса. Два самостоятельных вычислителя исполняют партии через *Parallel.For*. Независимые *unittest* на *Python* 3.12 запускают реальные процессы. Предусмотрено также контейнерное развёртывание четырёх служб.

Котировки заданием не предоставлены. Пользователь задаёт цены, прогноз, волатильности и корреляции; пример условный. Первая глава рассматривает методы, вторая – проектирование и реализацию *ASTER*, третья – верификацию и валидацию. В приложениях А и Б приведены испытания и листинг, в приложениях В, Г и Д – руководства пользователя, программиста и системного программиста.

1 ОБЗОР МЕТОДОВ И ПРОГРАММНО-ТЕХНИЧЕСКОГО ОБЕСПЕЧЕНИЯ

1.1 Подходы к оценке риска портфеля

Оценка риска начинается с определения измеряемой величины и горизонта. В данном проекте рассматривается изменение общей стоимости фиксированного набора длинных позиций за один год. Убытком считается разность начальной и конечной стоимости. При росте портфеля эта разность отрицательна. Такой выбор сохраняет связь показателя риска с денежными единицами и позволяет сопоставлять сценарии при одинаковой исходной стоимости. Для корректного сравнения все цены заранее выражаются в одной валюте.

Историческое моделирование применяет наблюдавшиеся изменения факторов к текущему портфелю. Для него необходимы согласованная история котировок, правила обработки пропусков и единые периоды наблюдения. Результат зависит от полноты истории и её соответствия рассматриваемому рыночному режиму. В исходном задании историческая выборка отсутствует, а прогноз и параметры передаются вместе с портфелем. Поэтому для данной задачи выбран расчёт по явно заданной вероятностной модели.

Параметрический подход описывает доходности и зависимости с помощью выбранного распределения и его параметров. Для простых линейных портфелей он позволяет быстро получить приближённые показатели. Однако усложнение функции стоимости и изменение сценарных предположений ограничивают удобство закрытых формул. Ошибка модели при этом сохраняется даже при точном вычислении формулы: математическая точность не подтверждает соответствие принятого распределения реальному рынку.

Моделирование Монте-Карло строит случайные реализации факторов и пересчитывает портфель для каждой из них. В изложении Г. Холтона этот подход связывает моделирование факторов с оценкой портфеля и получением распределения потерь [1]. Для курсового проекта важна независимость реализаций: их можно вычислять отдельными партиями и объединять в одну выборку. Число реализаций влияет на случайную погрешность эмпирического квантиля, а параметры модели определяют само распределение.

Сопоставление методов проводится по доступным данным и назначению расчёта. Исторический подход требует наблюдений, параметрический – подходящей формы распределения и функции стоимости, имитационный – проверяемой модели и достаточного числа реализаций. Для заданного портфеля исходными являются именно прогнозные параметры, поэтому отсутствие котировок не заменяется вымышленной историей. Монте-Карло позволяет исследовать эти параметры непосредственно; оценка остаётся условной относительно принятых предположений.

Качество результата рассматривается на двух уровнях. На уровне модели оцениваются правдоподобность параметров, постоянство зависимостей и соответствие ценового процесса выбранным инструментам. На уровне численного метода проверяются генерация выборки, вычисление стоимости и правило определения квантиля. Рост числа реализаций уменьшает влияние случайной выборочной ошибки, однако не устраняет ошибку предположений. Такое разделение необходимо при сравнении методов: быстрота исполнения и математическая проверяемость не означают достоверного прогноза будущего рынка.

1.2 Стрессовые сценарии и обоснование метода

Стресс-тестирование дополняет вероятностную оценку явным изменением предположений. В программе стресс задаётся начальным ценовым шоком по типу инструмента, общим сдвигом годового прогноза и множителем волатильности. Это делает условия эксперимента прозрачными: пользователь видит введённые проценты и может сохранить модель вместе с результатом. Названия предустановок описывают учебные сценарии, а не вероятность конкретного экономического события.

VaR не является верхней границей убытка. При уровне 95 процентов остаётся хвост худших исходов примерно в пять процентов выборки. Два распределения могут иметь одинаковый порог, но различаться тяжестью этого хвоста. Поэтому *ASTER* дополнительно выводит *ES95* по принятому правилу включения порогового ранга, вероятность положительной потери и крайние наблюдения. Эти величины интерпретируются совместно; одиночный показатель не заменяет рассмотрение сценарных предположений и ограничений.

Метод Монте-Карло соответствует условиям задания: годовой прогноз, волатильности и корреляции определяют модель цен, независимые траектории формируют распределение потерь, а его квантиль задаёт *VaR*. Независимость траекторий позволяет распределить выборку по вычислителям. Управляющий процесс должен принять полный объём потерь и лишь после этого определить показатели риска. Принятое решение объединяет проверяемую математическую модель и схему *master-worker*; его реализация рассмотрена во второй главе.

В вероятностном расчёте частота исхода определяется выбранной моделью. Стрессовое условие, напротив, задаётся как отдельное воздействие и не получает автоматически вероятности события. Поэтому результат базового эксперимента и результат сценария падения сравнивают при одинаковом портфеле, горизонте и правиле измерения потерь. Если одновременно изменены несколько факторов, различие итоговых показателей характеризует их совместное действие.

Денежный знак потери сохраняется во всех сценариях. Отрицательный *VaR* означает, что даже соответствующий выборочный порог относится к приросту стоимости; заменять его нулём означало бы изменить определение величины. При интерпретации полезны средняя потеря, доля положительных потерь и *ES95*. Для небольшой выборки хвост содержит мало наблюдений, поэтому увеличение объёма уточняет оценку, но не исправляет неверно выбранные прогнозные параметры.

Обоснование метода включает возможность независимой проверки. Детерминированные условия с нулевой волатильностью дают аналитический результат, а одиночный актив позволяет сравнить выборочные характеристики с логнормальным распределением. Сортировка полученных потерь делает ранг *VaR* проверяемым без повторения численного ядра. Эти свойства позволяют отделить ошибки программной реализации от ограничений финансовой модели и организовать верификацию в третьей главе.

1.3 Геометрическое броуновское движение и зависимости

Геометрическое броуновское движение, или *GBM*, описывает положительную цену через экспоненту случайного процесса. В отличие от простого прибавления нормального приращения к цене, экспоненциальное обновление не создаёт отрицательных цен при конечных параметрах. Работа К. Фати Ваджарга и М. Шоги рассматривает моделирование такого процесса и применение к оценке риска [2]. В проекте используется обычная псевдослучайная выборка, а не последовательности квазимонте-карло из исследованного авторами варианта.

Годовой прогноз в интерфейсе означает параметр дрейфа цены, а не гарантированную годовую доходность. Волатильность задаёт масштаб случайной компоненты. Оба параметра постоянны внутри одного эксперимента. Разделение дрейфа и волатильности позволяет описать неблагоприятный сценарий двумя независимыми воздействиями: уменьшить ожидаемое изменение стоимости и одновременно увеличить разброс возможных исходов. Начальный шок учитывается отдельно, чтобы его потери не исчезли при отсчёте от уже изменённой стоимости.

Моделирование каждого актива независимо игнорировало бы совместное движение цен. Для учёта зависимостей применяется корреляционная матрица. Её диагональные элементы равны единице, симметричные элементы совпадают, а значения находятся в диапазоне от минус единицы до единицы. Эти условия необходимы, но недостаточны: матрица должна быть положительно полуопределённой. Например, три попарных коэффициента допустимого диапазона всё равно могут оказаться взаимно несовместимыми.

Для построения коррелированного вектора применяется разложение Холецкого: нижняя треугольная матрица L удовлетворяет равенству $C=LL^{T}$. Умножение L на вектор независимых стандартных нормальных величин даёт приращения с заданными зависимостями. Вырожденные положительно полуопределённые матрицы требуют обработки нулевых диагональных элементов. Численный допуск позволяет учитывать погрешность операций с плавающей точкой; математически несовместимая матрица должна быть отклонена.

Число шагов определяет количество промежуточных точек траектории и вычислительную нагрузку. Для постоянных коэффициентов применяется точное экспоненциальное переходное соотношение *GBM*. Поэтому годовое распределение не требует приближения Эйлера с возможной отрицательной ценой. Однако при одинаковом начальном числе генератора варианты с 12 и 252 шагами потребляют разные последовательности случайных величин. Их выборочные результаты не обязаны совпадать, хотя аналитическая модель конечной цены одинакова.

Для акций, облигаций и валюты используется единое ценовое соотношение. Такое упрощение позволяет выполнить распределённый расчёт по заданию, но ограничивает финансовую интерпретацию. Облигации не содержат купонов и срока погашения; валютная позиция уже оценена в валюте портфеля. Нельзя утверждать, что реализация оценивает кредитный риск облигации или отдельно моделирует пару обменных курсов. Для расширения модели потребуются новые входные данные, функции стоимости и независимые проверки.

1.4 Технологии распределённого вычисления

Архитектура *master-worker* отделяет управление заданием от исполнения отдельных партий. Управляющий процесс знает полный объём выборки, назначает диапазоны индексов и агрегирует результаты. Вычислителю достаточно параметров модели, начала диапазона и количества траекторий. Отсутствие общей изменяемой математической структуры между вычислителями уменьшает число межпроцессных зависимостей. Центральный контроль полноты при этом становится обязательным условием публикации итогового результата.

Стандартные классы *.NET TcpListener* и *TcpClient* предоставляют средства *TCP*-взаимодействия, а *NetworkStream* – поток чтения и записи [3]. *TCP* сохраняет порядок байтов, но не границы отправленных *JSON*-объектов. Прикладной протокол должен самостоятельно передавать длину сообщения и читать ровно заданное число байтов. Однократное чтение из сокета не является надёжным способом получения всего запроса, поскольку данные могут приходить несколькими фрагментами.

Task Parallel Library предоставляет задачи и средства параллельной обработки [4]. Асинхронные задачи подходят для ожидания сетевых ответов, а

Parallel.For – для независимых вычислений на процессорных потоках. Эти уровни параллелизма можно сочетать: одновременно обращаться к нескольким вычислителям и внутри каждого ограниченно исполнять траектории. Ограничение числа потоков позволяет задать расход ресурсов и сравнивать конфигурации при одинаковых параметрах модели.

System.Text.Json входит в стандартную библиотеку *.NET* и предоставляет сериализацию структурированных данных [5]. *JSON* удобен для передачи параметров, сохранения модели и анализа обмена. Корректность сообщений требует отдельных правил: проверки типов и полей, ограничений глубины и объёма, контроля версии и идентификатора. Для *TCP* дополнительно необходимо кадрирование по длине, поскольку границы *JSON*-объектов не следуют из границ сетевых чтений.

Браузерный интерфейс может быть отделён от численного ядра *HTTP*-шлюзом. *HTML* задаёт структуру страницы, *CSS* – оформление, *JavaScript* – обработку действий. *HttpListener* позволяет обслуживать статические файлы и прикладные маршруты средствами *.NET*. Шлюз передаёт модель управляющему процессу через *TCP*. Такое разделение сохраняет единый вычислительный протокол для графического интерфейса и независимого клиента.

Для независимой проверки выбран *unittest* стандартной библиотеки *Python* 3.12 [7]. Проверочный клиент обращается к публичному протоколу через сокеты, а аналитические формулы и выгруженные потери служат эталонами. Отдельные процессы и временные каталоги позволяют проверить перезапуск и повреждение хранилища. Оформление пояснительной записки соответствует университетской методичке [8]; условия и фактические результаты испытаний вынесены в приложение А.

Контейнеры сохраняют ту же архитектуру: четыре службы общаются по *TCP*. Упаковка не меняет *GBM* и соглашение о ранге; меняются адреса и способ запуска.

Применение только стандартных библиотек уменьшает число внешних зависимостей учебного проекта. При этом оно не освобождает от ограничений реализации: нет защищённого *TLS*-канала, индивидуальных учётных записей, автоматического обнаружения узлов и резервного управляющего процесса.

При выборе средств разработки учитываются возможности воспроизведения проекта на другом компьютере. Целевая версия среды исполнения задаёт набор доступных классов, а *SDK* дополнительно содержит компилятор и средства сборки. Подготовленная публикация и исходный проект поэтому требуют разных компонентов. Контейнерная упаковка фиксирует среду приложения в образе; решение *Visual Studio* сохраняет структуру исходников и параметры запуска для доработки. Оба способа должны опираться на один исходный код.

2 ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ ПРОГРАММЫ

2.1 Архитектура, структура и исходные данные

Разработанное приложение *ASTER* принимает портфель и параметры сценария, моделирует годовую стоимость и рассчитывает показатели критических потерь. Управляющий процесс распределяет траектории между самостоятельными вычислителями и принимает индивидуальные убытки. Вычислители используют несколько потоков. Пользователь получает показатели риска, гистограмму, образцы траекторий и отчёт с исходными условиями; успешный результат требует полного запрошенного объёма выборки.

Портфель содержит от одного до восьми активов. Для каждого задаются уникальное название, тип, начальная цена, количество, годовой дрейф и годовая волатильность. Предусмотрены типы *stock, bond* и *currency*, которые определяют применение начального стрессового шока. Положительные цена и количество означают только длинные позиции. Короткие продажи и финансовый рычаг не входят в принятую модель; отрицательное количество не допускается.

Корреляционная матрица имеет размер, равный числу активов. Порядок её строк и столбцов совпадает с порядком активов. При изменении состава формы матрица перестраивается, но сервер всё равно проверяет размеры и значения независимо от браузера. Проверка не ограничивается визуальной симметрией. Пользователь, отправляющий *JSON* напрямую, проходит те же математические ограничения, что и пользователь графического интерфейса.

Число траекторий находится в диапазоне от 100 до 500 000. Интерфейс предлагает основные значения 10 000, 100 000, 200 000 и 500 000. Число шагов выбирается из 12, 52 и 252, горизонт фиксирован и равен году. Уровень доверия принимает значения 0,95 и 0,99. В отчёте сохраняются оба квантиля; выбор доверия определяет выделяемый основной показатель. Начальное число генератора служит частью воспроизводимой модели.

Сценарий задаёт название, общий сдвиг дрейфа, множитель волатильности и три начальных шока. Базовый сценарий имеет нулевые шоки и единичный множитель. Предустановка «Падение рынка» изменяет дрейф на минус 0,12, увеличивает волатильность в 1,7 раза, задаёт шоки акций минус 25 процентов, облигаций минус 8 процентов и валюты плюс 12 процентов. Эти параметры являются открыто указанными учебными условиями, а не оценкой текущего рынка.

Стоимость условного примера равна 100 000 *BYN*. Акции имеют цену 120 и количество 300; облигации – 1000 и 40; валютная позиция – 3,2 и 7500. Годовые дрейфы составляют 0,09, 0,055 и 0,025, волатильности – 0,24, 0,08 и 0,14. Попарные корреляции равны 0,2; минус 0,15; 0,05. Исходный файл

examples/portfolio.json используется и для демонстрации, и для контрольной апробации.

Проверка модели сосредоточена в классе *Validation* и выполняется до создания работы. Названия активов должны быть непустыми и уникальными без учёта регистра после удаления крайних пробелов. Цена и количество находятся в диапазоне от 10^{-6} до 10^{6}, исходная стоимость не превышает 10^{10}. Проверяются конечность чисел, допустимые типы, число шагов, уровень доверия и трёхбуквенный код валюты. Все цены заранее выражаются в этой валюте.

Матрица проверяется по размеру, диапазону коэффициентов, симметрии и единичной диагонали. Затем выполняется разложение, устанавливающее допустимость зависимостей. Те же правила применяются на вычислителе при приёме партии. Ошибочные значения возвращаются пользователю с указанием причины; автоматическое исправление корреляций или цен не производится. Сохранённая модель описывает именно принятые параметры эксперимента.

Расчёт признаётся завершённым только после принятия всех запрошенных потерь, вычисления сводки и успешной записи архива. Ошибка, отмена и перезапуск незавершённой работы имеют отдельные состояния. Показатели *VaR* для таких состояний отсутствуют. Дополнительно сохраняются время расчёта, обработанное количество по вычислителям и число сетевых повторов. Эти сведения позволяют отличить полный результат от сообщения о ходе выполнения.

На рисунке 2.1 показана архитектура приложения. Браузер передаёт параметры *HTTP*-шлюзу *Gateway*. Шлюз обращается по *TCP* к управляющему процессу *Master*, который принимает работу, назначает партии двум *Worker* и проверяет ответы. Сводка создаётся после получения всех потерь и сохранения результата. Вычислители запущены самостоятельными процессами операционной системы; прямой вызов их методов из *master* не используется.

Основной сценарий начинается с ввода или импорта портфеля. Пользователь задаёт активы, корреляции, прогноз и стресс, выбирает число траекторий и запускает расчёт. *Gateway* преобразует *HTTP*-запрос в команду *TCP*. *Master* проверяет модель, сохраняет начальное состояние и возвращает идентификатор работы. Браузер периодически запрашивает её состояние, поэтому длительный расчёт не удерживает один *HTTP*-запрос до завершения.

После расчёта пользователь просматривает показатели и выгружает отчёт. История позволяет открыть две завершённые записи для сравнения *VaR* и условий. Параметры сохраняются вместе с результатом, поэтому сравнение можно воспроизвести по данным моделей.

Администратор запускает роли приложения, задаёт адреса и количество потоков вычислителей, проверяет доступность узлов. В демонстрации запуск автоматизирован, но при работе на разных компьютерах роли запускаются отдельными командами. Настройка адресов относится к конфигурации процессов.

Она не изображается как функция редактирования сети внутри браузера. Отдельная диаграмма вариантов использования выполнена в физическом формате A1 и включена в графический материал.

Историю централизованно хранит *Master*. *Worker* получает модель, начало диапазона и количество траекторий; постоянного архива на вычислителе нет. Переназначение диапазона меняет исполнителя, сохраняя математические параметры. Список адресов вычислителей фиксируется при запуске управляющего процесса. Для новой конфигурации *Master* перезапускается с нужными *endpoint*; автоматического обнаружения узлов нет.

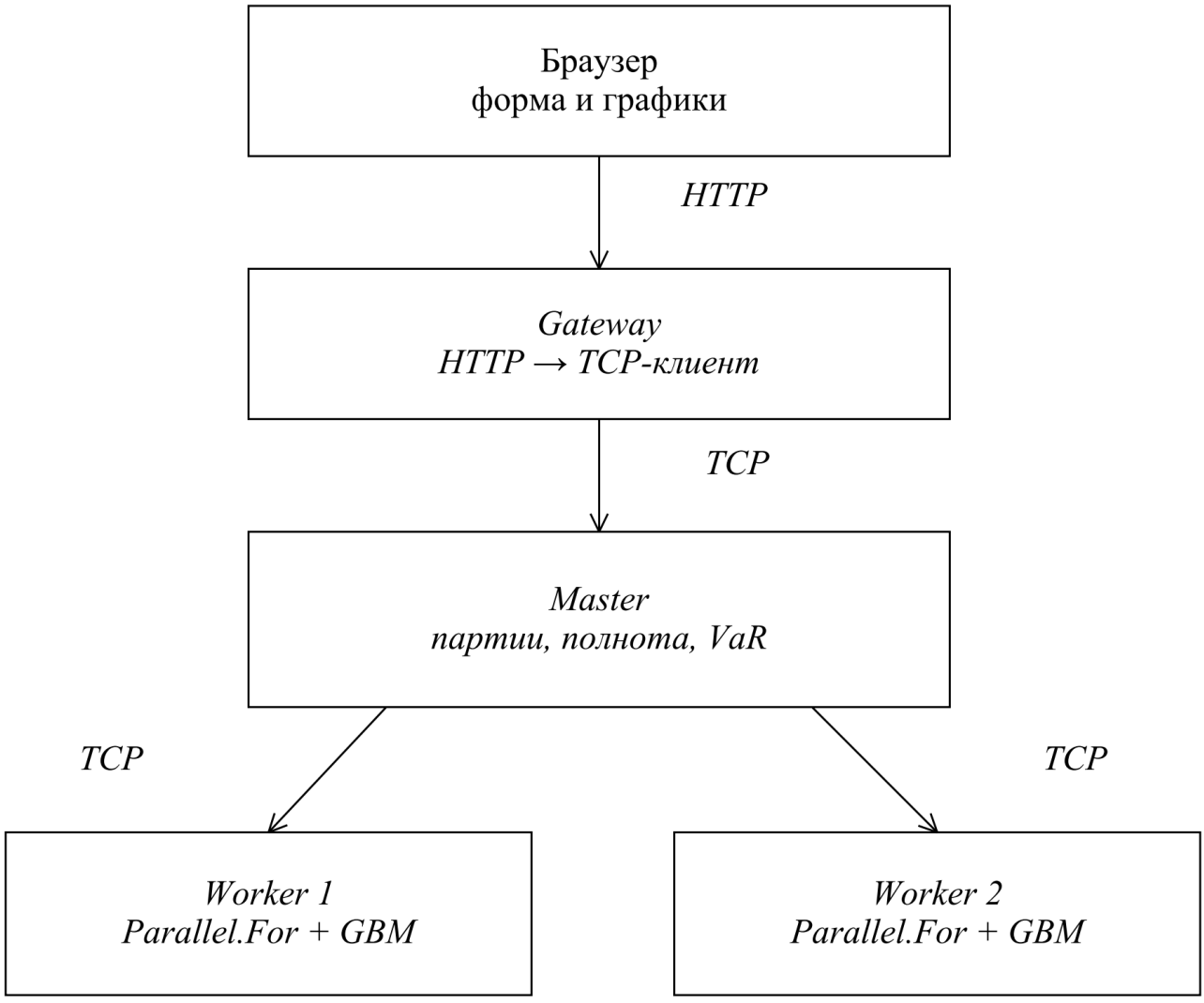


Рисунок 2.1 – Функциональная схема приложения

Проект *src/AsterRisk* использует целевую платформу *net8.0* и стандартную библиотеку *.NET* 8 без внешних пакетов *NuGet*. Исходники разделены на математические компоненты *Core*, сетевой слой *Network*, службы *Services* и браузерные файлы *wwwroot*. Изменение оформления не требует изменения численного ядра.

На рисунке 2.2 показаны классы и структуры данных. *Gateway* обслуживает *HTTP* и выгрузки, *Wire* выполняет кадрирование и сетевой вызов, *Master* назначает партии и хранит состояния работ. *Worker* принимает вычислительную команду и вызывает *Engine*. *Validation* проверяет модель и строит разложение Холецкого, *Store* сохраняет архив. Компоненты взаимодействуют посредством композиции и явно передаваемых объектов.

Asset описывает один актив, *Simulation* объединяет активы, корреляции и параметры эксперимента. *Stress* хранит сценарные воздействия, *Batch* – диапазон траекторий. *BatchResult* возвращает потери и образцы. *Result* представляет итоговую сводку, *JobView* – состояние логической работы и её идентификаторы. Неизменяемые записи удобны для сериализации и передачи между слоями, а *mutable*-коллекции работ защищены общей блокировкой управляющего процесса.

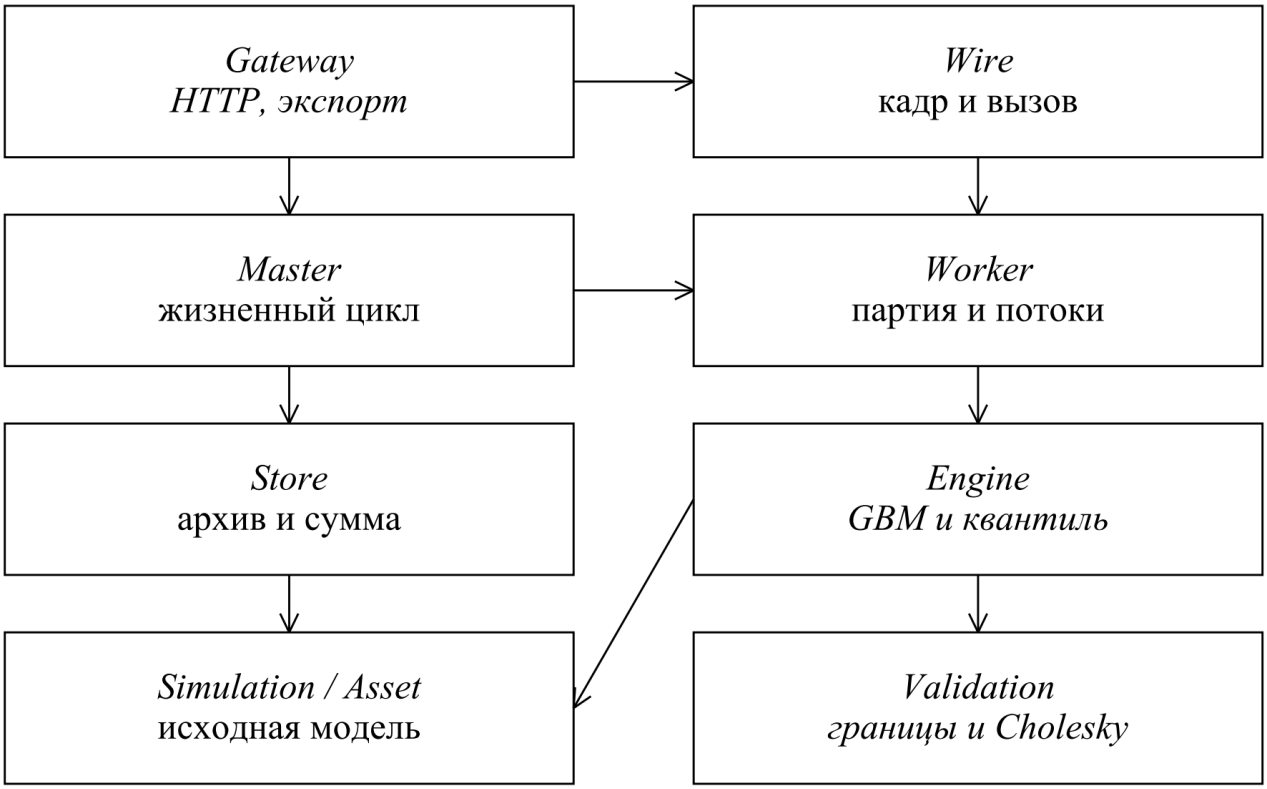


Рисунок 2.2 – Взаимодействие классов и структур данных

Engine.cs содержит генератор, моделирование и агрегирование, *Models.cs* – сериализуемые записи, *Validation.cs* – ограничения и разложение матрицы. Эти компоненты не обращаются к браузеру или архиву. *Engine* принимает *Batch* и возвращает *BatchResult* с потерями и образцами.

Wire.cs реализует общий обмен для *Master* и *Worker*. *Master.cs* содержит назначение диапазонов и состояния, *Worker.cs* связывает *calculate* с ядром,

Store.cs сохраняет историю. *Gateway.cs* предоставляет *HTTP*-маршруты и экспорт, обращаясь к *Master* по *TCP*.

Program.cs разбирает роль и создаёт службы. *Worker, master* и *web* запускаются отдельно, *demo* создаёт дочерние процессы. *Calculate* рассчитывает одну диагностическую партию до 5000 траекторий; полный эксперимент выполняется через *Master. Tests* и *tools* содержат проверки и средства воспроизведения.

В *wwwroot* находятся *index.html, style.css* и *app.js:* структура формы, обе темы и обработка действий. *JavaScript* переводит процентные значения в доли, отправляет запросы и отображает состояния. Пользовательский текст выводится средствами *DOM* без исполнения *HTML.*

2.2 Реализация математических алгоритмов

Начальная стоимость портфеля вычисляется как сумма произведений цены на количество по всем активам. До применения случайных приращений цена каждого инструмента умножается на единицу плюс соответствующий начальный шок. Для валюты, акции и облигации выбирается собственный сценарный коэффициент. Исходная стоимость для определения потери сохраняется до шока. Поэтому мгновенное падение цены включается в итоговую годовую потерю.

$$V_0 = \sum_i q_i S_{i,0}; \quad S_{i,\text{нач}} = S_{i,0}(1 + h_i). \tag{2.1}$$

В формуле 2.1 q_i – количество актива; $S_{i,0}$ – исходная цена; h_i – начальный шок. Горизонт разделяется на m шагов, и величина Δt равна $1/m$ года. Скорректированная волатильность σ'_i равна исходной σ_i, умноженной на сценарный множитель. Скорректированный дрейф равен сумме исходного μ_i и общего сдвига a. Все параметры выражены в долях единицы, хотя форма показывает проценты.

$$S_{i,k+1} = S_{i,k}\, exp[(\mu_i + a - \sigma'^{\,2}_i/2)\Delta t + \sigma'_i\sqrt{\Delta t} z_{i,k}]. \tag{2.2}$$

Для получения z на каждом шаге формируется независимый нормальный вектор ε, после чего выполняется умножение $z=L\varepsilon$. Матрица L строится из корреляционной матрицы C так, чтобы $C=LL^{T}$. Нижняя треугольная структура позволяет вычислять каждый компонент без обращения к элементам выше диагонали. Для восьми активов стоимость этого преобразования мала относительно сотен тысяч временных траекторий, поэтому достаточно собственной реализации на массивах *double.*

Разложение начинается с первой строки и последовательно вычисляет диагональные и поддиагональные элементы. От каждого элемента исходной матрицы вычитается сумма уже построенных произведений. Отрицательный диагональный остаток ниже допуска означает недопустимость корреляций. Малый нулевой остаток допускается для вырожденной матрицы, но ненулевой несовместимый поддиагональный остаток отклоняется. Валидация применяется как на *master*, так и на *worker*.

Engine заранее рассчитывает постоянные коэффициенты дрейфа и волатильности. Каждая траектория использует собственные массивы цен и приращений. На шаге формируется нормальный вектор, умножается на L и обновляет цены. Отмена проверяется внутри цикла; неконечная потеря вызывает ошибку партии.

Псевдослучайные числа создаёт 64-разрядный генератор *SplitMix64*. Реализация преобразования состояния соответствует опубликованному алгоритму, приведённому в материалах С. Виньи [6]. Равномерные значения формируются из старших битов, затем формула Бокса–Мюллера получает стандартную нормальную величину. Генератор не применяется для паролей или ключей доступа. Полного статистического аудита и доказательства неперекрытия всех подпотоков в рамках проекта нет.

Начальное состояние каждой траектории определяется общим *seed* и её абсолютным индексом. *Engine* сочетает *seed* с произведением фиксированной 64-разрядной константы и значения *index+1*. Генератор создаётся внутри обработки траектории, поэтому потоки не используют общее изменяемое состояние. Перенос партии на другой *Worker* сохраняет индекс и последовательность случайных величин.

Parallel.For обрабатывает позиции массива партии с ограничением *MaxDegreeOfParallelism*. Число вычислительных потоков задаётся при запуске *Worker* и ограничивается диапазоном от 1 до 16. Каждый поток записывает потерю в свою позицию массива. Порядок завершения траекторий не влияет на их принадлежность диапазону; *Master* позже помещает принятую партию в соответствующую область общей выборки.

После последнего шага вычисляется V_t как сумма конечных цен, умноженных на неизменное количество. Потеря $\ell = V_0 - V_t$ записывается в локальный массив партии. Первые 64 траектории дополнительно сохраняют последовательность промежуточных стоимостей для графика. Остальные траектории возвращают только конечную потерю, что сокращает объём сетевых ответов. График 64 линий служит иллюстрацией и не является доверительным интервалом прогноза.

Воспроизведение требует одинаковой модели, числа шагов и *seed*. Изменение шагов меняет расход случайных чисел. Номер *Worker* в состояние генератора не входит. Совпадение при смене конфигурации проверено в одной сре-

де *.NET*; другая платформа может отличаться последними битами математических функций.

Перед агрегированием управляющий процесс проверяет длину принятого массива и конечность каждой потери. Условие полного количества является необходимым: систематически потерянные партии могут содержать худшие исходы и исказить хвост распределения. Только после принятия всех диапазонов массив сортируется по возрастанию. Порядок завершения сетевых задач не влияет на отсортированную выборку и на используемый способ определения ранга.

$$k_{\alpha} = \lceil \alpha N \rceil; \quad VaR_{\alpha} = \ell_{(k\alpha)}. \tag{2.3}$$

Здесь N – число траекторий, α – уровень доверия, а $\ell_{(k\alpha)}$ – элемент с указанным порядковым рангом, начиная с единицы. В коде индекс массива равен $k\alpha - 1$. Интерполяция между соседними рангами не выполняется. Для *N=100* 000 квантиль 95 процентов соответствует рангу 95 000, квантиль 99 процентов – рангу 99 000. При экспорте *CSV* сохраняется именно отсортированный массив потерь, поэтому этот расчёт можно проверить независимо.

Если выбранный ранг имеет отрицательную потерю, *VaR* также остаётся отрицательным. Программа не заменяет его нулём и не скрывает знак. Такой результат означает прибыль на соответствующем модельном ранге, но не гарантирует отсутствия худших исходов. Для корректного чтения интерфейс отдельно поясняет знак потери и связь *VaR* с эмпирическим квантилем. Денежные значения округляются только при отображении, исходная точность хранится в отчёте.

Средний хвостовой убыток *ES95* определяется как арифметическое среднее элементов от ранга $k_{0,95}$ до конца включительно. При *N=100* 000 в это среднее входит 5001 элемент, поскольку пороговый ранг включён. Это конкретное дискретное соглашение зафиксировано в документации. Оно не должно незаметно подменяться средним строго худших пяти процентов при сравнении с другим инструментом. Ранговый тест независимо проверяет сортировку экспорта и оба уровня *VaR*; правило *ES* зафиксировано в коде агрегирования.

Вероятность убытка вычисляется как доля потерь строго больше нуля. Средняя потеря определяется по всей выборке, крайние наблюдения – по её первому и последнему элементам после сортировки. Для гистограммы диапазон делится на 40 интервалов; максимум включается в последний. При одинаковых потерях все наблюдения относятся к первому интервалу, поэтому деление на ноль не выполняется. Отображение столбцов обрабатывается отдельно в браузере.

Сводка *Result* содержит исходную стоимость, оба *VaR*, *ES95*, вероятность убытка, гистограмму и первые 64 образца. Она также включает длительность

расчёта и обработанное количество по узлам. Длительность отсчитывает *Master* от начала исполнения работы до агрегирования; время открытия браузера в неё не входит. Округление применяется к представлению, а в сохранённом *JSON* остаются исходные значения *double.*

2.3 Сетевое взаимодействие и хранение результатов

Выборка делится на последовательные диапазоны размером до 5000 траекторий. Для *N=100* 000 создаётся 20 партий, для *N=500* 000 – 100 партий. Последняя партия может быть меньше остальных. Диапазоны не пересекаются и покрывают все абсолютные индексы. Первичное назначение выбирает вычислителей циклически. Для каждого *endpoint* установлен семафор, разрешающий исполнение одной партии одновременно, поэтому очередь работы не перегружает один *worker.*

Запрос *calculate* содержит *Simulation, start* и *count.* Ответ возвращает начало диапазона, массив *losses* и иллюстративные *samples*, если диапазон затрагивает первые 64 индекса. *Master* проверяет начало, точное количество чисел, конечность, допустимую верхнюю границу потери и структуру образцов. Граница потери связана с положительной конечной стоимостью длинного портфеля. Некорректный ответ не принимается как выполненная партия.

Каждый обмен использует кадр: четыре байта длины в порядке *big-endian* и затем *JSON* в *UTF-8.* Максимальный размер тела кадра – 4 *MiB.* Конверт содержит *version, key, id, command* и *data.* Ответ содержит ту же версию и идентификатор, флаг *ok* и данные либо описание ошибки. Идентификатор позволяет обнаружить ответ от другого запроса. На одном соединении предусмотрен один запрос и один ответ, после чего соединение закрывается.

Таблица 2.1 – Основные команды прикладного протокола

Команда	**Узел**	**Назначение**
ping	Все роли *TCP*	Проверка процесса и числа потоков
submit	*Master*	Приём модели и *UUID* работы
job / *history*	*Master*	Состояние и сохранённая сводка
cancel / *delete*	*Master*	Отмена либо удаление записи
losses	*Master*	Страница до 5000 потерь
calculate	*Worker*	Исполнение диапазона траекторий

При ошибке соединения партия пробуется на других *endpoint* не более одного раза на каждом. Поскольку поток случайных чисел зависит от абсолют-

ного индекса, повтор не меняет математические исходы. Если все попытки неуспешны, работа завершается ошибкой. Уже полученные потери не превращаются в успешную сводку. Не реализована схема бесконечного ожидания возвращения узла или автоматической репликации управляющего процесса.

Таймаут соединения составляет 3 секунды, чтения входного кадра – 5 секунд, клиентского вычислительного вызова – 120 секунд. Общий обработчик ограничен 130 секундами, а работа – 10 минутами. Отмена чтения *NetworkStream* может закрыть соединение раньше отправки *JSON*-ошибки. Клиент должен воспринимать закрытие и сброс соединения как отказ операции. Тест оборванного кадра проверяет отсутствие успешного ответа, а не обязательное наличие текста ошибки.

Для повторной отправки пользовательской работы применяется *requestId* в виде *UUID*. Одинаковый идентификатор с теми же сериализованными параметрами возвращает существующую работу. С другим набором параметров возникает конфликт. Эта идемпотентность действует на *master* и сохраняется в архиве. Идентификатор *TCP*-конверта относится к конкретному сетевому вызову, а *requestId* – к логической работе; эти назначения различаются.

Сетевые чтения выполняются до получения всей объявленной длины. Повторяющиеся *JSON*-ключи, неизвестные поля, чрезмерная глубина, неверная версия и ключ отклоняются до исполнения команды. *TCP*-сервер допускает не более 32 обработчиков. При достижении лимита новый обмен не создаёт неограниченное число задач. Общий ключ передаётся через окружение; он контролирует доступ к командам, но не шифрует канал.

Учёт принятых партий защищён блокировкой *Master*. После проверки ответа в массив копируется *count* потерь, начиная с индекса *start*, отмечается завершение партии и увеличивается счётчик принятых траекторий. Повторное получение уже принятого диапазона не добавляет вторую выборку. Перед агрегированием проверяются отметки всех партий и точное количество потерь. Поэтому успешная сводка относится ко всему запрошенному эксперименту.

На рисунке 2.3 приведён жизненный цикл работы. Новая работа сначала сохраняется в состоянии *running*, затем становится *completed*, *failed* либо *cancelled*. При загрузке сохранённого *running* после остановки процесса назначается *interrupted*. Автоматического продолжения нет: пользователю следует создать новую работу и сохранить прежний протокол ошибки. Сводка появляется только у *completed* после успешного сохранения полного результата.

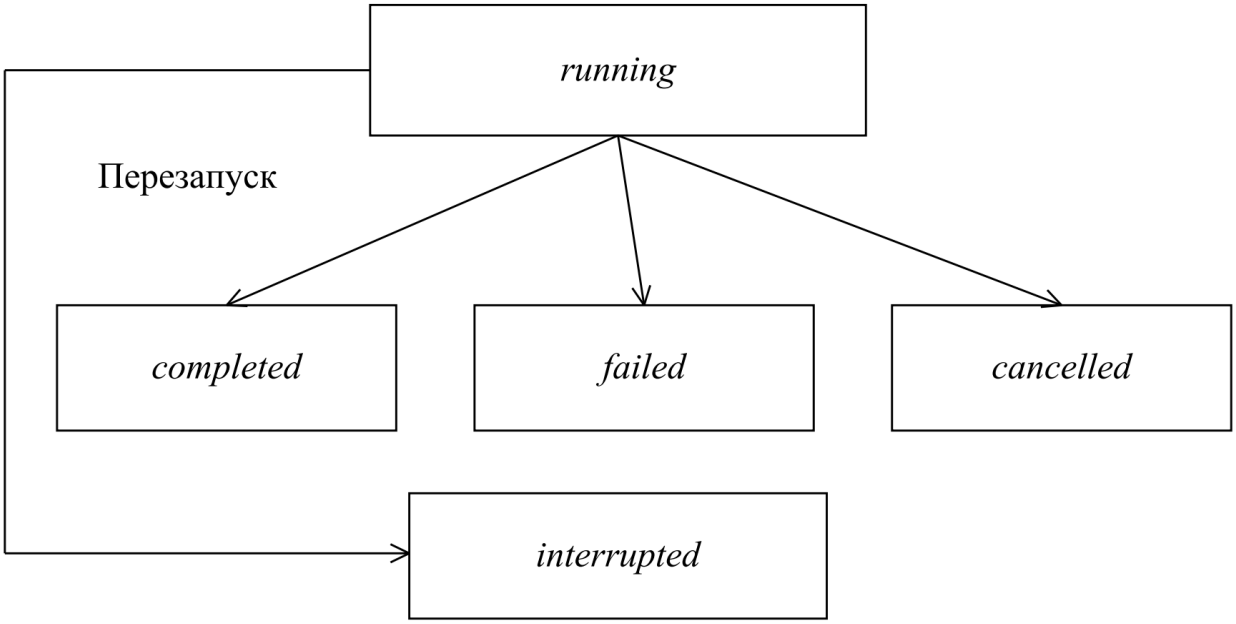


Рисунок 2.3 – Жизненный цикл вычислительной работы

Приём *submit* создаёт *JobView* с новым идентификатором и сохраняет состояние *running* до начала исполнения. После завершения всех партий *Master* сначала записывает полный результат и потери через *Store*, затем публикует *completed* в памяти. Ошибка сохранения не превращается в успешный расчёт. Команда *cancel* устанавливает сигнал отмены; *failed* и *cancelled* сохраняются без итоговой сводки.

Архив содержит *JSON*-записи с контрольной суммой *SHA-256*. Сумма вычисляется по сериализованному содержимому и проверяется перед загрузкой. Запись создаётся во временном файле, данные сбрасываются *Flush*(*true*), после чего выполняется замена целевого файла. Класс *FileStream* поддерживает файловый обмен и режимы доступа [10]. Эксклюзивно открытый *master.lock* запрещает второй управляющий процесс на том же каталоге. При повреждении записи запуск прекращается с ошибкой вместо молчаливой потери части истории.

При загрузке *Store* проверяет имя файла и идентификаторы, допустимое состояние, параметры модели и полноту завершённого результата. Архивная запись ограничена 20 *MiB*. Если контрольная сумма или структура ошибочна, запуск завершается диагностикой. После восстановления записи *running Master* меняет её состояние на *interrupted* и сохраняет это изменение. Для повторного математического исполнения пользователь создаёт новую работу.

RequestId сохраняется вместе с моделью и восстанавливается после перезапуска. Повтор прежнего *UUID* возвращает существующую запись, позволяя

получить потерянный ответ. Для нового исполнения той же модели создаётся новый *UUID*. Таким образом архив поддерживает идемпотентность логической работы.

Master одновременно допускает две активные работы и хранит не более двадцати записей. При заполнении истории удаляется ненужная завершённая запись; корзина отсутствует. Один массив из 500000 значений *double* занимает около 4 *MiB* без накладных расходов. Лимиты ограничивают расход памяти и диска.

Резервная копия создаётся после остановки *Master* и включает весь каталог архива. В контейнерном варианте данные находятся в именованном томе. Проверка *SHA-256* обнаруживает случайное изменение содержимого; защита от подделки и проверка внезапного отключения питания не выполнялись. При повреждении сохраняется исходный каталог для анализа, а восстановление производится из копии. Изменение суммы вручную не восстанавливает потерянные данные.

2.4 Запуск процессов и графический интерфейс

3 ВЕРИФИКАЦИЯ И ВАЛИДАЦИЯ ПРОГРАММЫ

3.1 Верификация математических алгоритмов

3.2 Верификация сетевого обмена и хранения

3.3 Валидация пользовательских и отказных сценариев

3.4 Апробация приложения и анализ измерений

ЗАКЛЮЧЕНИЕ

Список использованных источников

1. Holton, G. A. Monte Carlo Value-at-Risk / G. A. Holton // Value-at-Risk: Theory and Practice. – 2nd ed. – 2014. – URL: https://www.value-at-risk.net/monte-carlo-value-at-risk/ (дата обращения: 09.10.2026).

2. Fathi Vajargah, K. Simulation of Stochastic differential equation of geometric Brownian motion by quasi-Monte Carlo method and its application in prediction of total index of stock market and value at risk / K. Fathi Vajargah, M. Shoghi // Mathematical Sciences. – 2015. – Vol. 9. – P. 115–125. – DOI: 10.1007/s40096-015-0158-5. – URL: https://link.springer.com/article/10.1007/s40096-015-0158-5 (дата обращения: 09.10.2026).

3. Use TcpClient and TcpListener / Microsoft // .NET documentation. – URL: https://learn.microsoft.com/en-us/dotnet/fundamentals/networking/sockets/tcp-classes (дата обращения: 09.10.2026).

4. Task Parallel Library (TPL) / Microsoft // .NET documentation. – URL: https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/task-parallel-library-tpl (дата обращения: 09.10.2026).

5. Serialize and deserialize JSON using C# / Microsoft // .NET documentation. – URL: https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview (дата обращения: 09.10.2026).

6. Vigna, S. A PRNG shootout / S. Vigna. – URL: https://prng.di.unimi.it/ (дата обращения: 09.10.2026).

7. unittest – Unit testing framework / Python Software Foundation // Python 3.12 documentation. – URL: https://docs.python.org/3.12/library/unittest.html (дата обращения: 09.10.2026).

8. Трохова, Т. А. Дипломное проектирование : учеб.-метод. пособие / Т. А. Трохова, И. А. Мурашко, К. С. Курочка. – Гомель : ГГТУ им. П. О. Сухого, 2019.

9. How Compose works / Docker Inc. // Docker Documentation. – URL: https://docs.docker.com/compose/intro/compose-application-model/ (дата обращения: 09.10.2026).

10. FileStream Class / Microsoft // .NET API reference. – URL: https://learn.microsoft.com/en-us/dotnet/api/system.io.filestream?view=net-8.0 (дата обращения: 09.10.2026).

11. Tutorial: Containerize a .NET app / Microsoft // .NET documentation. – URL: https://learn.microsoft.com/en-us/dotnet/core/docker/build-container (дата обращения: 09.10.2026).

ПРИЛОЖЕНИЕ А

(обязательное)

Условия и результаты испытаний

ПРИЛОЖЕНИЕ Б

(обязательное)

Листинг программы и тестов

ПРИЛОЖЕНИЕ В

(обязательное)

Руководство пользователя